AI理解、引擎驱动：零代码搭建实时数据链路

直播技术团队 大淘宝技术 2026年8月18日 23:31 浙江 7人

直播数据团队创新性地提出"指标驱动"的实时数据开发范式，将复杂的流式计算简化为"维度+指标"的业务逻辑表达。用户仅需200字符内的自然语言描述，系统即可自动识别指标维度，通过AI辅助生成DSL并转换为Flink SQL任务，开发周期从天级缩短至分钟级。系统核心是指标驱动引擎，能自动回溯依赖关系构建链路拓扑，并采用"理解与正确性分离"架构确保可靠性，通过Hook机制支持安全增量调整。该设计屏蔽了流式数据复杂性，让开发聚焦业务本身，同时沉淀可复用指标资产，形成良性循环，显著提升实时数据研发效率。

01 背景：实时数据开发为何"重"

在直播业务中，存在大量实时数据的应用场景：对内的直播 360 大促分析，对外的主播数据大屏、实时榜单等数据产品，以及嵌入业务主链路的实时数据工程链路（平台策略调控、实时算法特征、广告投放）。

离线数据开发的核心是**业务逻辑的正确表达**；而实时数据开发在此之上，还必须处理流式数据的复杂特性——窗口设计、双流关联语义、状态生命周期管理、时间属性与乱序处理等，才能完成对无限、动态流的状态化持续计算。因此，相较离线，实时数据链路的开发门槛更高，代码评审与纠错成本也更高。

那么，面向这些场景，我们能否抽象出一套更高效、更轻量的研发模式？能否通过产品化抽象，**隔离流式数据的复杂特性处理，让实时开发重新聚焦业务逻辑本身**？

我们的切入点是：对于数据链路，业务逻辑的核心可以抽象为"在**哪个维度**上计算出**什么指标**"，即**维度 + 指标**。观测和追踪**指标的实时变化**正是实时数据的核心价值，指标的计算过程撑起了整个数据任务的骨架。沿着这一思路，我们探索出一套 **AI 辅助、指标驱动**的实时数据端到端开发系统。

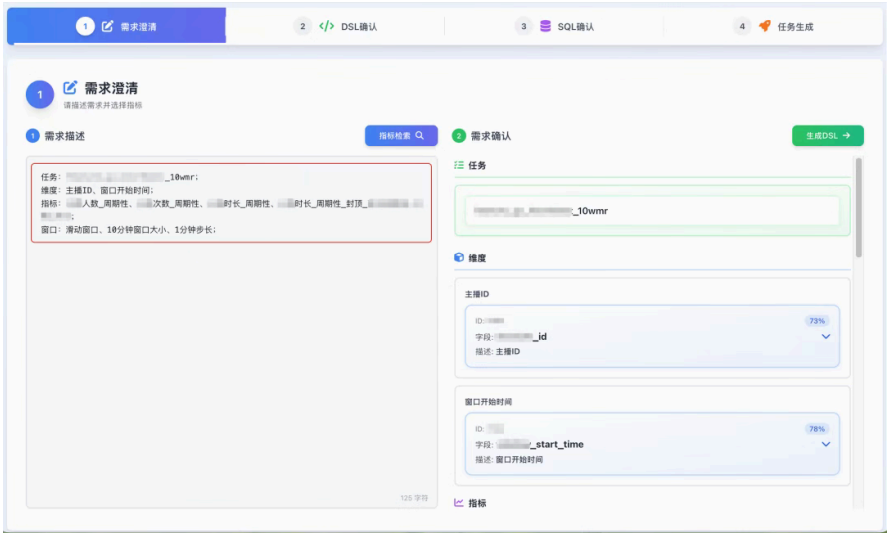
本文将从一个具体案例切入，介绍直播数据团队在搭建该系统过程中的实践与思考。

02 最佳应用案例

我们以一个典型的实时特征数据需求为例，从**用户视角**完整走一遍使用链路——看看从一句自然语言需求，到一个可发布的 Flink SQL 任务，中间经历了什么。

2.1 需求描述

- **需求澄清**：用户用自然语言描述需求，系统自动完成指标的检索召回，以及窗口等逻辑的识别处理。
- **需求确认**：用户确认指标、维度、窗口的处理结果符合预期后，系统自动生成 DSL。







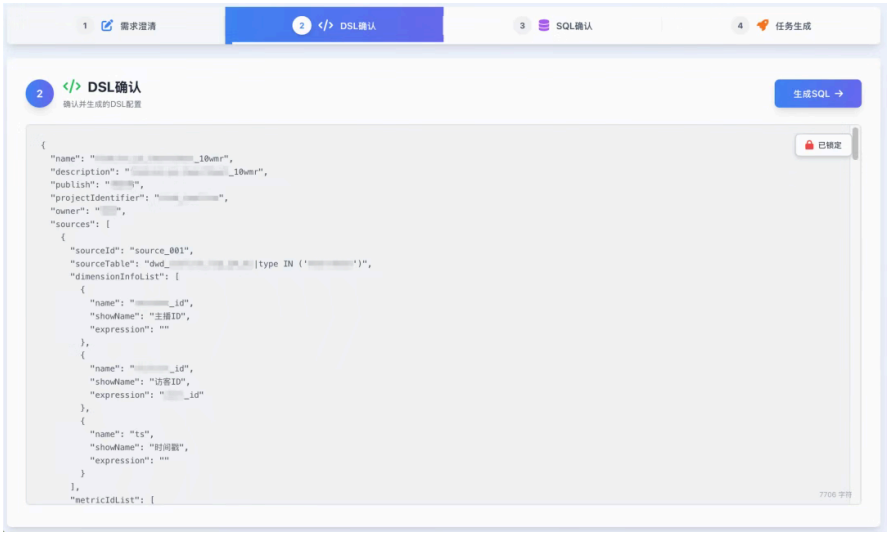
（上图红框部分是用户手动输入内容，其余是系统生成）

如此轻量的需求描述，得益于**指标驱动**的架构设计（系统如何据此自动构建链路，详见第三章）。这一步的本质，是把用户以**自然语言描述的需求**，转换为系统可理解的**结构化技术需求**。

2.2 DSL 确认

用户简单浏览 DSL（无需强校验，为方便排查暂保留此步），点击确认后系统自动生成 SQL。

这里的 DSL 是贯穿前端、AI、后端三方的**共识契约**——AI 生成它、前端编辑它、后端消费它（设计理念见方法论 4.3）。系统在这一步已完成从**"技术需求"**到**"技术实现方案"**的转化，其内部的构建过程会在第三章详细展开。



（系统生成DSL）

2.3 SQL 生成

系统间通过 DSL 协议通信，工程化地将技术方案转化为 SQL 代码，确保 100% 准确。



（系统生成SQL）

至此，若 SQL 已满足全部需求，即可一键生成 Flink SQL 任务、跳转 Dataphin 平台发布 🎉；若还需局部调整，则进入下一步的增量需求分析 👇。

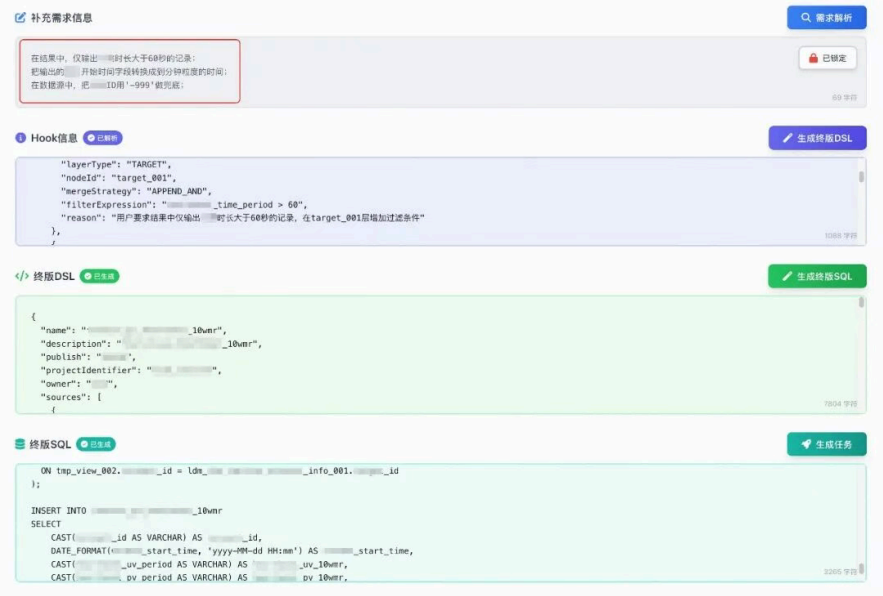
2.4 增量调整（可选）

拿到初版 SQL 后，用户判断是否还需要局部调整：

- 若已满足全部需求 → 直接一键生成 Flink SQL 任务、跳转 Dataphin 平台发布 🎉；
- 若还需局部调整（例如"在某个临时视图中增加一个过滤条件"）→ 用自然语言描述调整点，系统据此**增量更新链路**并重新生成 SQL，确认无误后再发布。

增量调整背后的机制——如何在不改变链路拓扑的前提下安全地打补丁——涉及 Hook 协议与围栏校验，详见第三章 3.4。

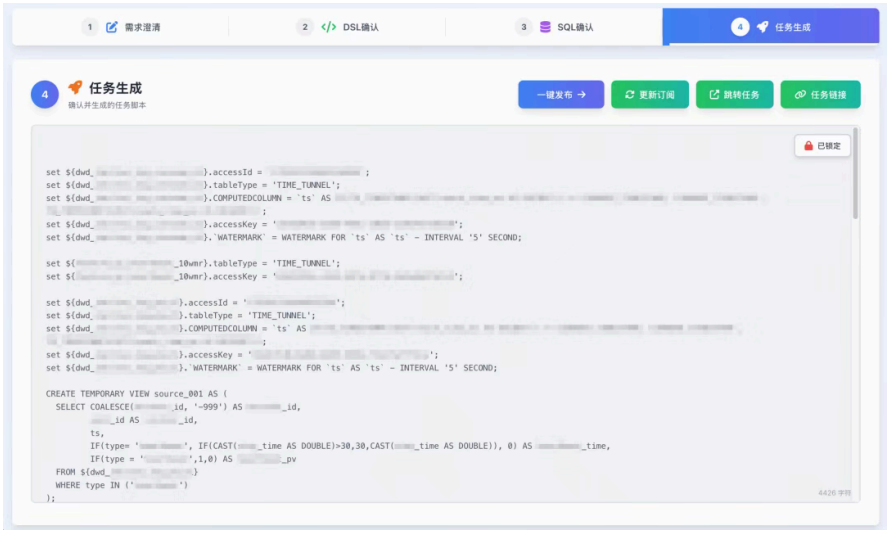
回到本案例中，用户通过简要的自然语言描述，补充了三个调整需求：对输出结果做筛选，对输出维度做格式转换，和对数据源中的维度做兜底处理。大模型会解析这部分**增量需求**，并转化成对应的**增量DSL协议**（Hook结构体），与已有的初版DSL进行融合。用户可直接在终版SQL中确认这三个增量的调整逻辑实现是否符合预期。



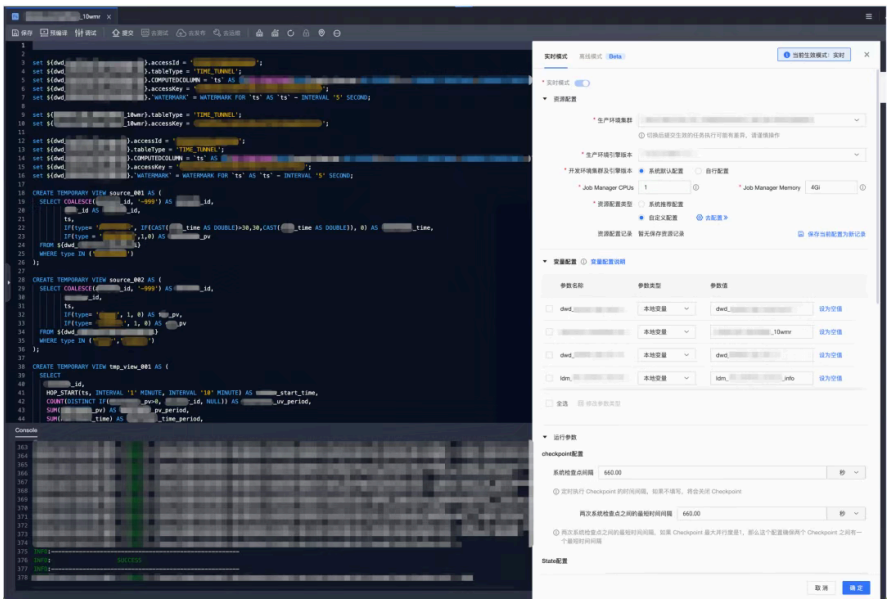
（上图红框部分是用户手动输入内容，其余是系统生成）

2.5 任务发布

系统自动生成Dataphin的Flink SQL任务脚本（包含对 TT Topic 的订阅处理，任务的资源、变量、参数信息配置）后，用户可以一键发布任务到Dataphin平台上。



（实时任务脚本）



（任务一键发布到Dataphin平台）

2.6 小结

在这个例子的整个开发链路中，需要用户**手动输入的需求描述**总计在**200个字符以内**。整个流程中的执行和决策是分开的：**人负责决策**，AI加持的系统负责执行。任务的**开发周期从天级缩短至分钟级**，显著提升了整体研发效率。

03 链路的构建与演进

第二章展示了"用户使用的操作流程"，本章深入系统内部：先看指标驱动引擎如何把一份结构化需求**自动构建**成初版链路（3.1—3.3），再看链路如何在初版基础上**增量演进**（3.4）。



3.1 引擎的两个核心动作

- **依赖回溯建骨架**：系统从"最终要产出的指标"出发，依据指标间的依赖关系，自主回溯所有过程指标，自动构建出整个任务的 DAG 拓扑骨架。用户只需"声明要什么指标"，无需"编写怎么算"。
- **维度双路填充**：维度信息分两路填充——被指标引用的**过程维度**，通过与指标的关联关系补全；作为聚合粒度的**输出维度**，则从产出结果一路向上回溯到源。二者分治，互不干扰。

3.2 链路分层与指标回溯图

下图是本任务经引擎回溯后生成的完整链路拓扑：



可以看到，引擎在拓扑生成阶段**统观全局、分而治之**，整条链路并不是系统预置的固定模板，而是引擎在理解目标指标后，基于指标依赖关系自底向上回溯、**动态生成出来任务的拓扑结构**。也就是说，source 从哪些物理表来、中间需要生成多少个 tmp_view、每个 tmp_view 分别承担什么加工职责、最终如何汇合到 target，都不是提前写死的，而是由目标指标的计算依赖、窗口语义、字段派生、聚合加工、维表关联以及输出适配需求共同决定。

在这个例子中，引擎识别到目标指标同时依赖心跳日志、消费日志和主播维表，并且中间存在行级派生、窗口聚合、封顶修正、维表关联和复合条件加工等不同类型的处理逻辑，最终把整条链路**动态拆解为清晰的「源 → 派生 → 聚合 → 加工 → 关联 → 产出」** 的分层结构，**每层职责单一、边界清晰**：

视图层	核心职责	加工性质
物理源表	原始日志接入 + 过滤（`type IN (...)`）	数据入口
源字段派生层（source）	行级表达式派生（`IF(type=...)`）	无状态映射
窗口聚合层（tmp_view_001）	HOP 窗口 + `GROUP BY` 聚合	有状态聚合
封顶加工层（tmp_view_002）	阈值封顶（`>300`）+ 直通透传	后聚合修正
维表关联层（tmp_view_003）	LOOKUP 维表 + 条件复合	跨源汇合
产出层（target）	类型转换（`CAST AS VARCHAR`）	输出适配

这套分层结构的价值在于：**引擎能够根据指标依赖自动决定链路形态**：

- 简单指标可能只需要 source → target，
- 窗口类指标需要生成 source → tmp_view → target，
- 而跨源复合指标则可能继续扩展出多个 tmp_view、多条 source 支路以及维表关联层。

最终生成的不是一套静态模板，而是一份围绕目标指标动态组织出来的、职责清晰且可部署的实时计算方案。

3.3 链路解读

结合上面的案例，有四个值得关注的机制：

1. **直通型 vs 复合型**：
   a. `xxx_pv/time/uv_10wmr` 三个基础指标自聚合层产出后层层直通、中间不加工；
   b. `xxx_time_10wmr_cap_xxx` 是**多数据源 + 维表**的复合型指标；
   c. 在 `tmp_view_003` 完成三路依赖汇合，代表了系统"多级派生 + 跨源条件筛选"的能力。
2. **扇出复用**：
   a. `xxx_time_period` 既直通产出基础指标，又向上参与封顶与复合加工，一次聚合、多路消费，避免重复计算。
3. **维表旁路增强**：
   a. `ldm_xxx_info` 仅在 `tmp_view_003` 通过 LOOKUP 横向"贴"入以提供 `xxx_level`，是旁路增强而非主链路加工。
4. **全链路血缘**：
   a. 任意产出指标都可**单向回溯到确定的物理源表与原始字段**，无论加工逻辑多复杂，产出与源之间始终保持一条清晰、确定、可审计的回溯路径：

```
xxx_time_10wmr_cap_xxx
  ├─ xxx_time  ← dwd_{source a}_.xxx_time
  ├─ xxx_pv/xxx_pv   ← dwd_{source b}_di.type
  └─ xxx_level   ← ldm_xxx_info（维表）
```

3.4 链路的增量演进：Hook 机制

初版链路生成后，系统支持在**不改变逻辑拓扑结构**的前提下做局部调整。这一能力回答了 2.4 中"增量调整背后发生了什么"。

① 交互点设计：为什么放在 SQL 生成之后？为什么要二次需求描述？

从体检来看：在与用户的人工交互环节，我们优先保证 human-friendly 的体验，让交互节点契合人对信息理解、处理的习惯。例如"在某个过程指标生成后注入一个过滤条件"——若前置到最初的需求描述，用户很难用简洁语言来准确描述条件的注入点位；而在生成初版 SQL 后，用户可直接参照 SQL 结构描述："在 `tmp_view_xxx` 中，增加 xx 指标大于 0 的过滤"。

从架构分层来看：增量需求的使用场景，解决的是单任务层面的一次性的调整逻辑处理。这部分的处理逻辑，是面向具体应用场景（ADS层）高度个性化的诉求，**不具备跨任务间的复用价值**。而初始阶段的需求澄清，聚焦的则是**高复用性的核心指标、维度资产**的召回，**以及窗口的制式化解决方案**。两类需求的处理方式，边界清晰。

② 适用场景

增量演进面向不改变任务逻辑拓扑结构的局部调整，例如：

精炼口径：在某个临时视图中增加筛选过滤条件；

维度调整：调整输出维度的展示（如时间格式），或增加兜底值处理；

输出冗余信息：输出时增加 `ext` 字段承载非核心字段（处理时间戳、过程指标值等监控/调试信息）。

③ 结构化协议：Hook

秉持最小化改动原则，我们设计了"增量补丁"协议 —— **Hook**。AI 只需输出结构化的修改指令，由后端引擎执行与校验。通过对 `hookType` 与 `mergeStrategy` 的抽象，Hook 成为扩展性极高的可插拔"外挂"（其设计理念见方法论 4.4）：

在增量需求分析环节，我们通过LLM能力进行需求分析，核心是要降低模型幻觉。通过对关键信息的裁剪，严格**控制了给模型输入的上下文长度** -- 只包含 **初版DSL + Hook增量协议**；其他冗长的内容（初始需求的分析、指标口径、SQL），则不必在此赘述给模型。

```
{
"hooks":[
{
"hookType":"FILTER | DIMENSION_TRANSFORM",
"layerType":"SOURCE | VIEW | TARGET",
"nodeId":"source_001 | tmp_view_001 | target_001",
"mergeStrategy":"REPLACE | APPEND_AND | APPEND_OR",
```

```
  "filterExpression":"仅 FILTER 类型时填写",
  "dimensionName":"仅 DIMENSION_TRANSFORM 类型时填写",
  "expression":"仅 DIMENSION_TRANSFORM 类型时填写",
  "reason":"简要说明为什么这样做"
  }
  ]
  }
```

④ 安全围栏校验（双保险）

前校验：应用 Hook 前，校验Hook结构体的准确性与完整性（类型、层级、节点、必填字段是否合法）；

后校验：更新 DSL 后，校验DSL其结构完整性、依赖关系与拓扑逻辑的准确性。

两道校验前后夹击，确保任何一次增量演进都**不会破坏链路的自洽性**。经增量演进生成终版 SQL、用户确认无误后，即可一键生成 Flink SQL 任务并跳转 Dataphin 发布。



第二、三章展示了"系统怎么用、怎么实现"，本章会介绍"系统为什么这样设计"。以下五点，是我们在实践中提炼出的、可迁移到更多场景的设计原则。

4.1 指标驱动作为核心范式

核心洞察：数据链路的业务逻辑可归约为"维度 + 指标"，而**指标之间的依赖关系天然构成一张有向无环图（DAG）**。因此，只要以"最终要产出的指标"为驱动，系统就能自动回溯出完整的计算拓扑（其工作机制详见第三章）。

这一范式的价值在于：它把"写 SQL"转化为"声明指标"，将流式数据的窗口、状态、乱序等复杂度封装在引擎内部，让用户的聚焦回归业务语义本身。指标既是开发的入口，也是驱动整个链路自动成形的核心动力——这是一种**声明式的实时数据开发范式**。

4.2 「理解」与「正确性」的分离架构

核心洞察：整个系统最本质的架构决策，是**让 LLM 负责"理解"，让确定性代码负责"正确性"**。二者职责边界清晰、互不越权：

- **AI Agent 只做翻译**（自然语言 → 结构化意图），不为输出的合法性背书；

- **后端引擎做执行 + 校验**（围栏机制），保证最终产出一定合法；
- **前端做人工确认**（Human-in-the-Loop），弥补两者之间的认知 gap。

这种分离带来的直接收益是：LLM 的"幻觉"被限制在"理解层"，无法穿透到"正确性层"污染最终产物。任何 AI 的误解，都会在确定性引擎的校验环节被拦截或被人工确认环节修正。这个架构模式，对所有"LLM 落地到生产系统"的场景都有借鉴意义——**不要让概率模型直接产出需要确定性保证的结果，而要让它产出"意图"，再由确定性系统兑现"结果"**。

4.3 DSL 作为多系统协同的"万能合约"

核心洞察：DSL 在系统中同时扮演三重角色——它既是**存储格式**，也是**传输协议**，还是**校验对象**。前端、AI、后端三方都通过同一份 DSL 对齐语义：

- AI 生成的结果，最终要能映射为 DSL 的字段；
- 前端编辑的，也是 DSL；
- 后端消费的，还是 DSL（DSL → SQL）；
- 而 Hook 机制，本质就是"DSL 的增量补丁协议"。

其设计难点在于：**如何让一份 DSL 同时满足"人可读"、"AI 可生成"、"机器可执行"三重约束**。我们的取舍是——字段语义贴近业务（人可读）、结构规整且枚举收敛（AI 可生成、幻觉率低）、层级与依赖显式化（机器可执行、可校验）。正是这份"万能合约"，让三个异构系统得以低耦合协同。

4.4 Hook 协议 —— LLM 安全接入确定性系统的工程范式

核心洞察：让 LLM 直接修改一份复杂的 DSL JSON 很不可靠（结构易错、难校验、出错只能重来）；但让 LLM 输出一个"增量操作指令"（Hook），再由确定性引擎执行，就变成了一种可控范式。

- **为什么不让 LLM 直接输出终版 DSL**：DSL 动辄数百行、字段间存在关联约束，LLM 难以保证全局一致性，且无法做增量校验；
- **Hook 协议的三要素设计**：`hookType`（做什么，如筛选过滤 / 维度变换）、`layerType + nodeId`（在哪做，精确定位到某层某节点）、`mergeStrategy`（怎么合并，且 / 或 / 替换）。三者正交组合，覆盖绝大多数调整场景；
- **围栏校验的双保险**：前校验保证指令本身合法，后校验保证应用后拓扑仍然自洽。在当前大模型能力下，Hook 生成准确率约 97%+，而双校验消除剩余 3% 的不确定性，使终版 SQL 完全可信；
- **可推广性**："LLM 输出结构化指令 → 确定性引擎执行 → 围栏校验兜底"这一三段式，可迁移到任何"需要用自然语言驱动、但要求结果确定可靠"的系统改造场景。

同时，Hook 的可扩展性很强：`hookType` 未来可扩展多路输出、新增维度等；`mergeStrategy` 也可灵活演进。它让系统的能力边界成为**"可插拔"**的。

4.5 实时资产沉淀路径

核心洞察：直播实时数仓有其特性——公共层通常仅沉淀 DWD 明细层，几乎没有 DWS 层，因为实时数据的时间特性处理使得 DWS 的复用效率不高。这意味着传统"靠 DWS 做厚中间层"的资产沉淀路径在实时场景下失效。

我们的解法，是通过这套**指标驱动**的方式，连通业务语义与指标资产——把散落在各任务中的指标显式化、标准化、可复用，从而**做厚"指标资产"这一层**。指标既是开发的驱动力，也成为沉淀下来的核心资产，**形成"用得越多、沉淀越厚、复用越易"的正循环**。

05 未来规划

1. **全链路能力 Skill 化封装**：当前系统页面对人工校验更友好；未来随安全围栏校验增强，逐步弱化人工校验强度，让整个流程更适配对话式交互。
2. **Agent 驱动的资产复用**：自动化需求分析与指标配置链路，替代手动的资产录入和检索流程。
3. **任务迭代与版本管控**：在"创建新任务"之外，支持已有任务的迭代与版本管理。
4. **Hook 类型扩展支持拓扑变更**：支持会改变拓扑结构的复杂增量需求，如多路输出、生成依赖于指标的维度。
5. **支持依赖于指标的维度**：某些维度不参与指标计算，但其生成逻辑依赖任务中的某个过程指标（如按指标数量级生成分层维度）。这类场景在实时任务中虽少见，但会打破当前的拓扑假设，需进一步扩展设计。

06 团队介绍

本文作者曦溪，来自淘天集团–直播技术数据团队。我们是一支在 AI 赛道上高速成长的技术先锋队，致力于用大数据与人工智能重塑数据生产关系。在直播场景中，团队率先走通了 AI Native 的数据研发范式，实现研发全流程的高质高效与数据的高可信；同时沉淀出面向 AI 的数仓规范、可信数据语义层与知识底座，将数据研发能力抽象为可复用的智能体资产，并进一步把 AI 从取数工具升级为懂业务的分析伙伴，推动数据消费从"提需求取数"走向"与 AI 对话做分析"，服务直播全域产运团队，实现规模化的认知普惠与确定性的数据增长。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

966篇原创内容

公众号

阅读 2068

留言

写留言

还没有留言，来写首评